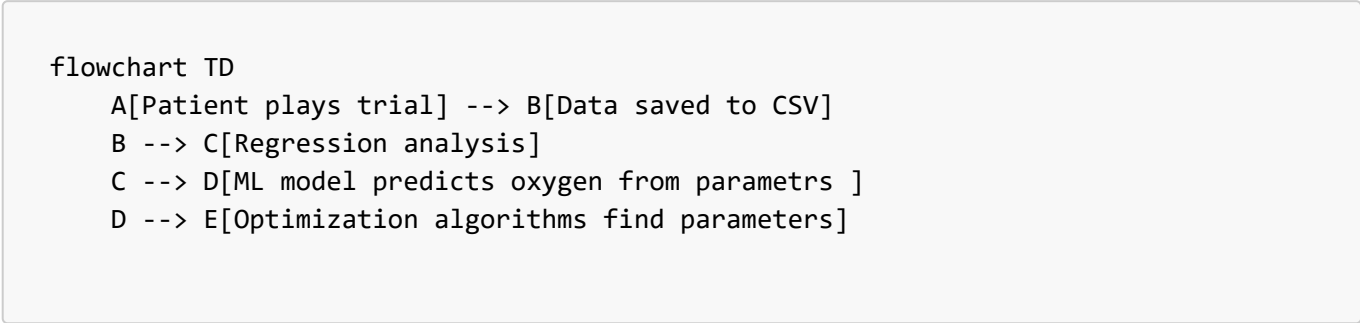


AquaGuardian Trial System - Overview

System Architecture

Data Flow Overview



Key Components

Core ML Components

1. TrialRegressionAlgorithm.cs - Main Orchestrator

- **Entry point:** `PerformRegressionAnalysis(allTrialData, targetOxygen)`
- **Role:** Coordinates the entire ML pipeline from data validation to report generation
- **Flow:** Validates data → trains model → runs cross-validation → optimizes parameters → generates reports
- **Location:** `Assets/Scripts/TrialRegressionAlgorithm.cs`

2. FeatureExtractor.cs - Feature Engineering

- **Key function:** `ExtractFeatures(TrialData)` returns `float[10]`
- **Role:** Converts raw trial data (CSV) into ML-ready features
- **Special handling:**
 - Adjusts `idleUpwardSpeed` for Amadeo mode ($\times 0.5$)
 - Calculates derived feature `EffectiveDrainRate`
 - Provides patient baseline via `GetPatientBaseline()` (uses median for personalization)
- **Why critical:** Proper feature extraction ensures model accuracy
- **Location:** `Assets/Scripts/Linear regression/FeatureExtractor.cs`

3. OxygenPredictor.cs - ML Model Wrapper

- **Key function:** `TrainModel(trials, enableFeatureSelection)`
- **Role:** Wraps the regression algorithm, handles training, prediction, and feature selection
- **Adaptive behavior:**
 - Adjusts Ridge λ based on dataset size (fewer trials = more regularization)
 - Can enable feature selection for small datasets (< 10 trials)
- **Interface:** `PredictOxygen(parameters)` returns predicted oxygen %
- **Location:** `Assets/Scripts/Linear regression/OxygenPredictor.cs`

4. MultipleLinearRegression.cs - ML Algorithm Core

- **Algorithm:** Ordinary Least Squares (OLS) with Ridge regularization
- **Matrix operation:** $\beta = (X^T X + \lambda I)^{-1} X^T y$
- **Role:**
 - Learns coefficients β_i (impact of each parameter on oxygen)
 - Handles z-score normalization
 - Computes feature importance
 - Performs K-Fold cross-validation
- **Why Ridge:** Prevents matrix singularity with small datasets, reduces overfitting
- **Location:** `Assets/Scripts/Linear regression/MultipleLinearRegression.cs`

5. DifficultyParameterSolver.cs - Optimization Algorithms

- **Core methods:**
 - `SolveForTargetOxygen()` - Primary method (3-phase: analytical + gradient)
 - `RandomSweepOptimizer()` - Fallback (random search)
 - `SolveForTargetDifficultyMulti()` - Final fallback (conservative gradient descent)
- **Role:** Finds game parameters that achieve target oxygen level (default 10%)
- **Innovation:** Multi-stage optimization with automatic fallback ensures robust solutions
- **Location:** `Assets/Scripts/Linear regression/DifficultyParameterSolver.cs`

6. RegressionUtilities.cs - Helper Utilities

- **Key functions:**
 - `OptimizeParameters()` - Orchestrates 3-stage optimization cascade
 - `PerformCrossValidationAndErrorCalculation()` - K-Fold CV
 - `ConstrainRangesToObserved()` - Dynamic parameter range adjustment (safety)
 - `PrepareOptimizationFeatures()` - Bans derived features, ranks by importance
- **Role:** Provides reusable utility functions for statistics, validation, and optimization
- **Location:** `Assets/Scripts/RegressionUtilities.cs`

Data & Reporting

7. TrialDataModels.cs - Data Structures

- **Models:**
 - `TrialData` - Contains 9 game parameters + results (oxygen %)
 - `RegressionResult` - Full analysis output (coefficients, metrics, optimized params)
 - `ParameterRanges` - Min/max bounds for each parameter
 - `CaveInfo` - Cave geometry data
- **Role:** Standardized data transfer between components
- **Location:** `Assets/Scripts/TrialDataModels.cs`

8. TrialReportGenerator.cs - Report Generation

- **Functions:**
 - `GenerateSummaryReport()` - Short report (optimized params, error)
 - `GenerateFullReport()` - Detailed report (coefficients, CV metrics, feature importance)
 - `SaveToFile()` - Saves to `RegressionResults/RegressionReport_YYYYMMDD_HHMMSS.txt`

- **Role:** Creates human-readable analysis reports
- **Location:** `Assets/Scripts/TrialReportGenerator.cs`

Game Integration

9. TrialParameterManager.cs - Parameter Management

- **Functions:**
 - Loads parameters from CSV (`Trial_5_runs_.csv` or `Trial_Random_Parameters.csv`)
 - Applies parameters to game systems (PlayerMovement, PlayerLife, Health)
 - Saves trial results back to CSV
- **Role:** Bridge between CSV data and Unity game objects
- **Location:** `Assets/Scripts/Trial/TrialParameterManager.cs`

10. TrialSystemManager.cs - Trial Lifecycle

- **Role:** Manages trial start/end, collects results, triggers regression analysis
- **Flow:** Start trial → patient plays → end trial → save results → (optional) run regression
- **Location:** `Assets/Scripts/Trial/TrialSystemManager.cs`

Execution Flow: From Trial to Optimized Parameters

This section explains the complete execution flow through the three most critical files.

Entry Point: TrialRegressionAlgorithm.cs

File: `Assets/Scripts/TrialRegressionAlgorithm.cs` (131 lines)

Main Function: `PerformRegressionAnalysis(allTrialData, targetOxygen)`

Flow:

1. **Validation** (lines 29-39):

```
if (allTrialData.Count < 3) return error;
```

Requires minimum 3 trials for ML analysis.

2. **Statistics** (line 48):

```
RegressionUtilities.CalculateTrialStatistics(allTrialData, result);
```

Calculates average oxygen across all trials.

3. **Model Training** (lines 51-52):

```
var predictor = new OxygenPredictor { maxFeaturesForTraining = 10 };  
bool trained = predictor.TrainModel(allTrialData, enableFeatureSelection:  
false);
```

Trains Multiple Linear Regression model on all 10 features.

4. **Cross-Validation** (lines 88-89):

```
var (cvRmse, cvMae, cvR2, kFolds, avgError) =  
    RegressionUtilities.PerformCrossValidationAndErrorCalculation(...);
```

Validates model generalization using K-Fold CV.

5. **Feature Preparation** (line 91):

```
var (bannedFeatures, filteredImportance) =  
    RegressionUtilities.PrepareOptimizationFeatures(allTrialData,  
    predictor);
```

Bans derived features and ranks by importance.

6. **Parameter Optimization** (lines 93-96):

```
var (optimizedSolution, optimizedError) =  
    RegressionUtilities.OptimizeParameters(model, predictor, allTrialData,  
    ...);
```

This is where the magic happens - finds optimal parameters for target oxygen.

7. **Report Generation** (lines 101-106):

```
result.summaryText = TrialReportGenerator.GenerateSummaryReport(...);  
result.fullDetailsText = TrialReportGenerator.GenerateFullReport(...);
```

Creates human-readable reports.

8. **Save Results** (lines 117-127):

```
SaveRegressionResultsToFile(result, "RegressionResults");
```

Optimization Coordinator: RegressionUtilities.cs

File: `Assets/Scripts/RegressionUtilities.cs` (284 lines)

Critical Function: `OptimizeParameters()` (lines 114-166)

Purpose: Orchestrates the 3-stage optimization cascade with automatic fallbacks.

Flow:

1. Build Optimization Indices (line 119):

```
int[] topIndices = BuildOptimizationIndices(filteredImportance,
featureNames);
```

Converts feature importance to parameter indices, excludes index 9 (derived feature). Falls back to indices `{4, 5}` (`RemoveHealthEveryLifeTime`, `removeHealthWithCollide`) if empty.

2. Prepare Baseline (line 120):

```
var (ranges, baseline) = PrepareOptimizationBaseline(trials, targetOxygen);
```

- Calls `ConstrainRangesToObserved()` (lines 169-196) to dynamically adjust parameter ranges
- Uses `FeatureExtractor.GetPatientBaseline()` to get median values (personalized)
- **Why:** Starting from patient's typical performance ensures realistic solutions

3. Stage 1: Primary Method (lines 122-126):

```
var solution = DifficultyParameterSolver.SolveForTargetOxygen(
    model, baseline, topIndices, targetOxygen, ranges,
    d => predictor.PredictOxygenUnclamped(d), out float error);
```

Calls the **most accurate** optimization method (3-phase analytical + gradient).

4. Stage 2: Random Fallback (lines 127-138):

```
if (solution == null || error > RANDOM_SWEEP_THRESHOLD) {
    var (sweepCandidate, sweepError) =
        DifficultyParameterSolver.RandomSweepOptimizer(...);
    if (sweepError < error) {
        solution = sweepCandidate;
        error = sweepError;
    }
}
```

If primary fails or error $> 5\%$, tries random search (150 samples).

5. **Stage 3: Conservative Fallback** (lines 140-148):

```
if (solution == null || error > 5f) {
    solution = DifficultyParameterSolver.SolveForTargetDifficultyMulti(
        model, targetOxygen, topIndices.Length, baseline, ranges, ...);
    error = CalculateError(predictor, solution, targetOxygen);
}
```

If random sweep also fails, uses gradient descent on top 3 features (conservative but stable).

6. **Exception Handling** (lines 152-165): If any stage crashes, falls back directly to `SolveForTargetDifficultyMulti`.

Key Helper Functions:

- `ConstrainRangesToObserved()` (lines 169-196): **Safety mechanism** - prevents optimizer from suggesting unrealistic parameters never seen in training
- `BuildOptimizationIndices()` (lines 78-102): Converts feature names to indices for optimization
- `PrepareOptimizationBaseline()` (lines 269-277): Prepares ranges and baseline for optimization

Optimization Algorithms: DifficultyParameterSolver.cs

File: `Assets/Scripts/Linear regression/DifficultyParameterSolver.cs` (577 lines)

This file contains the **core optimization algorithms**. Called by `RegressionUtilities.OptimizeParameters()`.

Primary Method: `SolveForTargetOxygen` (lines 179-236)

Purpose: Find parameters that achieve target oxygen with **minimal changes** from baseline.

Phases:

Phase 1: `SolveMinimalChange()` (lines 246-307)

```
var x0 = SolveMinimalChange(model, baseParams, free, targetO2, bounds, ...);
```

Algorithm:

1. Calculate fixed contribution from non-optimized parameters (line 257):
$$\text{fixed} = \beta_0 + \sum_{i \notin \text{free}} \beta_i \cdot x_i$$
2. Solve analytically for free parameters (lines 285-288):
$$s = \frac{\text{target} - \text{fixed}}{\sum_{j \in \text{free}} \beta_j^2}$$

3. Update each free parameter (lines 290-304): $x_{j,\text{new}} = x_{j,\text{old}} + s \cdot \beta_j$ (in normalized space, then convert back to raw values)
4. Clamp to valid ranges

Why it works:

- **Mathematically optimal** (least-squares solution)
- **Minimizes L2 distance** from baseline
- **Clinically important:** Small changes = easier for patient to adapt

Phase 2: RefineProjectedGradient() (lines 314-362)

```
var x1 = RefineProjectedGradient(model, x0, free, targetO2, bounds, predictO2, ...);
```

Algorithm:

```
For each iteration (up to maxSteps):
1. Predict oxygen with current parameters
2. Calculate error = predicted - target
3. For each free feature:
  - gradient = coefficient  $\beta_i$ 
  - step = -learningRate  $\times$  error  $\times \beta_i / \Sigma(\beta^2)$ 
  - Update in normalized space
  - Convert to raw space and clamp
4. Stop if error < 0.1%
```

Adaptive Learning Rate (lines 335, 218-219):

- error $> 40\%$: lr = 0.7 (aggressive)
- error $> 20\%$: lr = 0.5
- error $> 10\%$: lr = 0.3
- error $\leq 10\%$: lr = 0.2 (fine-tuning)

Why needed: Phase 1 assumes unconstrained optimization, but parameters have bounds. When solution hits bounds (clamps), it becomes suboptimal. Gradient descent adjusts.

Phase 3: RefineProjectedGradientIterative() (lines 369-437)

```
if (finalError > 0.5f) {
  x1 = RefineProjectedGradientIterative(model, x1, free, targetO2, bounds, ...);
}
```

Only runs if Phase 2 error $> 0.5\%$.

Key difference: Tracks **best solution** across all iterations (Phase 2 only keeps last).

- Lines 390-394: Updates `bestParams` whenever error improves
- Line 436: Returns `bestParams` (not `cur`)

Why: Phase 2 might oscillate around target. Phase 3 remembers the best point found.

Fallback 1: RandomSweepOptimizer() (lines 530-573)

When Used: Primary method fails or error $> 5\%$.

Algorithm:

```
for (int i = 0; i < 150; i++) {
    // Sample random parameters from valid ranges
    candidate = SampleRandomParameters(ranges);

    // Predict oxygen
    predicted = predictFunc(candidate);
    error = |predicted - targetOxygen|;

    // Track best
    if (error < bestError) {
        bestCandidate = candidate;
        bestError = error;
    }
}
```

Trade-off: Less accurate ($\sim 5\%$ error) but **robust** to:

- Noisy data
 - Non-convex optimization landscapes
 - Bad starting points
-

Fallback 2: SolveForTargetDifficultyMulti() (lines 38-172)

When Used: Random sweep also fails or error $> 5\%$.

Algorithm: Conservative gradient descent on top $K=3$ features.

Key Differences from Primary:

1. **Feature Selection** (lines 65-81):
 - Optimizes only top 3 most important features
 - More conservative (fewer moving parts)
2. **Adaptive Configuration** (lines 93-95):
 - error $> 30\%$: $\text{lr}=0.3$, $\text{maxIter}=250$
 - error $> 15\%$: $\text{lr}=0.25$, $\text{maxIter}=200$

- else: $lr\$=0.2\$$, $maxIter\$=150\$$

3. **Gradient Descent** (lines 97-166): Similar to Phase 2, but operates on fewer features for stability.

Why different: When primary methods fail, problem is likely difficult. Optimizing fewer features reduces complexity and increases stability.

Summary: Complete Pipeline

```
TrialRegressionAlgorithm.PerformRegressionAnalysis()
↓
RegressionUtilities.OptimizeParameters()
↓
[Stage 1] DifficultyParameterSolver.SolveForTargetOxygen()
  ↓ Phase 1: SolveMinimalChange() - Analytical (instant, optimal)
  ↓ Phase 2: RefineProjectedGradient() - Adjust for bounds
  ↓ Phase 3: RefineProjectedGradientIterative() - Track best (if error > 0.5%)
  ↓ Result: Typically <1% error ✓

[If error > 5%]
  ↓ Stage 2: RandomSweepOptimizer() - 150 random samples
  ↓ Result: 1-5% error (robust fallback)

[If still error > 5%]
  ↓ Stage 3: SolveForTargetDifficultyMulti() - Conservative gradient (top 3
  features)
  ↓ Result: Best effort, stable solution
```

Key Insight: The system tries increasingly conservative methods until it finds a good solution. Primary method is fast and accurate. Fallbacks ensure robustness even with difficult data.

The 10 Features (Game Parameters)

The system uses 10 features to predict final oxygen percentage:

Base Parameters (9)

These are the actual game settings that can be directly controlled:

1. **speed** - Player horizontal forward speed (units/sec)
2. **verticalSpeed** - Player vertical movement speed (units/sec)
 - Controls up/down movement responsiveness
3. **idleUpwardSpeed** - Passive upward drift when no input (units/sec)
 - Amadeo mode: multiplied by $0.5\$$
4. **lifeTime** - Oxygen tank lifetime (seconds)

- Base duration before oxygen depletes completely

5. **RemoveHealthEveryLifeTime** - Health drain per cycle

- Amount of oxygen removed every **lifeTime** seconds
- Example: if **lifeTime** = 2s and **RemoveHealthEveryLifeTime** = 5%, player loses 5% oxygen every 2 seconds

6. **removeHealthWithCollide** - Collision damage (%)

- Oxygen lost when hitting walls/caves

7. **timeBetweenCollides** - Collision cooldown (seconds)

8. **healHealthPoint** - Health pack restoration (%)

- Oxygen restored by collecting health items

9. **factorForce** - Amadeo device force multiplier

- Controls input sensitivity for Amadeo device
- Set to 0 for keyboard mode
- *Input mode detection*: Only set if Amadeo mode was actively used during trial

Derived Feature (1)

This is calculated automatically from base parameters:

10. **EffectiveDrainRate** - Calculated drain rate per second

- Formula: $\text{EffectiveDrainRate} = \frac{\text{RemoveHealthEveryLifeTime}}{\text{lifeTime}}$
- Example: **RemoveHealthEveryLifeTime** = 5%, **lifeTime** = 2s gives $\text{EffectiveDrainRate} = 2.5\%/\text{sec}$
- *Note*: This feature is banned from optimization (derived from other features)

Input Mode Handling

The system adapts features based on input device:

- **Keyboard Mode** (**IsAmadeoMode** = 0):
 - **factorForce** = 0 (no force sensitivity)
 - **idleUpwardSpeed** used as-is
- **Amadeo Mode** (**IsAmadeoMode** = 1):
 - **factorForce** active (force multiplier)
 - **idleUpwardSpeed** $\times 0.5$ (weaker drift)
 - Only marked as Amadeo mode if device was **actually used** during trial (not keyboard fallback)

Code location: `Assets/Scripts/Linear regression/FeatureExtractor.cs` lines 37-56

Machine Learning Pipeline

1. Data Collection

Where: `Assets/Scripts/Trial/TrialParameterManager.cs`

- Parameters loaded from `Trial_5_runs_.csv` (or `Trial_Random_Parameters.csv` for random mode)
- During gameplay, oxygen level tracked
- At trial end, results saved back to CSV with final oxygen %

CSV Format:

```
trial_id,speed,verticalSpeed,...,factor_force,o2_run1,o2_run2,...,o2_run79
1,15,30,0.01,2,2,5,2,10,3,70.4,80,80,...,80
2,12,35,0.5,1.5,3.5,8,1,5,2.5,40.8,44,54.5,...,48
```

2. Feature Extraction

Where: `Assets/Scripts/Linear regression/FeatureExtractor.cs`

Critical Function: `ExtractFeatures(TrialData trial)` - Converts raw trial data to ML features

Amadeo Mode Adjustments:

```
bool isAmadeo = trial.IsAmadeoMode > 0.5f;
float idleUpSpeed = isAmadeo ? trial.idleUpwardSpeed * 0.5f :
trial.idleUpwardSpeed;
float factorForce = isAmadeo ? trial.factorForce : 0f;
```

Patient Baseline: `GetPatientBaseline(trials, ranges, useMedian=true)`

- ≥ 3 trials: uses **median** from patient history (personalized)
- < 3 trials: uses mid-range defaults
- Why median:** Robust to outliers, represents typical patient performance
- Critical for optimization:** Starting point must be realistic to patient's abilities

3. Model Training (CRITICAL COMPONENT)

Where: `Assets/Scripts/Linear regression/OxygenPredictor.cs`

Algorithm: Multiple Linear Regression with Ridge Regularization

Mathematical Model:
$$\text{finalOxygen} = \beta_0 + \beta_1 \cdot \text{speed} + \beta_2 \cdot \text{verticalSpeed} + \dots + \beta_9 \cdot \text{EffectiveDrainRate}$$

Each β_i represents the **impact** of parameter i on final oxygen percentage.

Key Implementation Details:

- Z-score Normalization** (critical for convergence):
$$\text{normalized} = \frac{\text{value} - \text{mean}}{\text{stdDev}}$$
 Ensures all features contribute equally regardless of scale.

2. Adaptive Ridge Regularization (prevents overfitting):

```
float ridgeLambda = Mathf.Clamp(0.5f + (10f - trialCount) * 0.2f, 0.5f, 2.0f);
```

- Fewer trials → higher λ → more regularization
- More trials → lower λ → less regularization
- **Why:** Small datasets prone to overfitting, Ridge adds penalty to large coefficients

3. Validation Thresholds:

- Large datasets (≥ 10 trials): $R^2 > 0.5$, error $< 15\%$
- Small datasets (< 10 trials): $R^2 > 0.2$, error $< 25\%$ (relaxed due to limited data)

4. Model Implementation (ML Core)

Where: `Assets/Scripts/Linear regression/MultipleLinearRegression.cs`

Matrix Equation (closed-form solution): $\beta = (X^T X + \lambda I)^{-1} X^T y$

Where:

- X = feature matrix ($n_{\text{trials}} \times n_{\text{features}}$)
- y = target vector (oxygen %)
- λI = Ridge penalty term (identity matrix scaled by λ)
- β = learned coefficients

Why Ridge: Without λI , matrix $(X^T X)$ may be **singular** (non-invertible) with small datasets. Ridge ensures invertibility.

Cross-Validation: K-Fold CV validates generalization

- $k=2$ for ≤ 5 trials
- $k=5$ for ≥ 10 trials
- Reports: RMSE (prediction error), MAE (average error), R^2 (variance explained)

Optimization Algorithms

The system uses a **multi-stage optimization strategy** to find parameters that achieve target oxygen level (default 10%).

Entry point: `RegressionUtilities.OptimizeParameters()` in `Assets/Scripts/RegressionUtilities.cs` lines 114-166

Strategy Overview

1. Try SolveForTargetOxygen (primary, analytical + gradient)
 - if fails or error $> 5\%$
2. Try RandomSweepOptimizer (random search fallback)

- if still fails or error $> 5\%$
- 3. Try SolveForTargetDifficultyMulti (gradient descent fallback)

Return best solution found

Algorithm 1: SolveForTargetOxygen (Primary Method)

Where: `Assets/Scripts/Linear regression/DifficultyParameterSolver.cs` lines 189-246

Importance: **Most accurate** - typically achieves $< 1\%$ error

Three-phase approach:

Phase 1: Minimal-Change Solver (Analytical) - MOST IMPORTANT

Method: `SolveMinimalChange()` lines 259-320

Goal: Find **minimal parameter changes** from patient's baseline to reach target oxygen.

Mathematical Foundation:

1. Separate fixed and free contributions:
$$\text{fixed} = \beta_0 + \sum_{i \in \text{non-free}} \beta_i \cdot x_i$$
$$\text{target} = \text{fixed} + \sum_{j \in \text{free}} \beta_j \cdot x_j$$
2. Find optimal scale factor (least-squares solution):
$$s = \frac{\text{target} - \text{fixed}}{\sum_{j \in \text{free}} \beta_j^2}$$
3. Update free parameters:
$$x_{j,\text{new}} = x_{j,\text{old}} + s \cdot \beta_j$$

Why This Works:

- **Minimizes L2 distance** from baseline (smallest parameter changes)
- **Closed-form solution** (no iteration, instant)
- **Respects gradient direction** (moves along steepest descent)
- **Clinically important:** Small changes = easier for patient to adapt

Example: Baseline predicts 35% , target is 10% (need to increase difficulty by 25%)

- $\beta_{\text{RemoveHealth}} = -8.2$ (large negative impact)
- $\beta_{\text{removeCollide}} = -3.5$ (moderate negative impact)

Solution: Increase RemoveHealth more than removeCollide (proportional to $|\beta|$) Result: Smooth difficulty increase, minimal disruption to patient experience

Phase 2: Projected Gradient Refinement

Method: `RefineProjectedGradient()` lines 329-377

Purpose: Refine Phase 1 solution to account for **parameter bounds**.

Why Needed: Phase 1 assumes unconstrained optimization, but parameters have bounds (e.g., speed $\in [10, 30]$). When Phase 1 solution hits bounds (clamps), it becomes **suboptimal**. Gradient descent adjusts.

Gradient Descent Step:
$$\text{step} = -\text{lr} \cdot \text{error} \cdot \frac{\beta_i}{\sum \beta^2}$$

Adaptive Learning Rate:

- Large error ($> 40\%$): $\text{lr} = 0.7$ (aggressive)
- Moderate error ($10\text{--}40\%$): $\text{lr} = 0.3\text{--}0.5$
- Small error ($< 10\%$): $\text{lr} = 0.2$ (fine-tuning)

Convergence: Stops when error $< 0.1\%$ (excellent accuracy)

Phase 3: Iterative Refinement (Rare Cases)

Method: `RefineProjectedGradientIterative()` - Only if Phase 2 error $> 0.5\%$

Key Difference: Tracks **best solution** across all iterations (Phase 2 only keeps last).

Why: Prevents local minima (Phase 2 might oscillate, Phase 3 remembers best point)

Algorithm 2: `RandomSweepOptimizer` (Fallback)

When Used: Primary method fails or error $> 5\%$

Algorithm: Generate $N=150$ random parameter combinations, return best.

Trade-off: Less accurate ($\sim 5\%$ error) but **robust** to noisy data, non-convex landscapes.

Algorithm 3: `SolveForTargetDifficultyMulti` (Final Fallback)

When Used: Random sweep also fails or error $> 5\%$

Difference from Primary:

- Optimizes only top $K=3$ features (more conservative)
- Lower learning rates ($0.2\text{--}0.3$)
- More stable for difficult cases

Complete Workflow

Entry Point

Where: `Assets/Scripts/TrialRegressionAlgorithm.cs` method `PerformRegressionAnalysis()`

```
// Main workflow
var result = TrialRegressionAlgorithm.PerformRegressionAnalysis(
    allTrialData,
    targetOxygen: 10f
);

// Returns: RegressionResult with optimized parameters
```

Step 1: Data Validation (lines 29-39)

Minimum \$3\$ trials required.

Step 3: Train Model (lines 50-79)

Uses all \$10\$ features with adaptive Ridge regularization.

K-fold CV ($k=2\text{--}5$), reports generalization metrics.

Step 6: Optimize Parameters (lines 93-96)

15 / 17

```
        model, predictor, allTrialData, filteredImportance,  
        FeatureExtractor.FeatureNames, targetOxygen, anyAmadeoTrials  
    );  
    // Three-stage optimization (see Optimization Algorithms section)
```

Finds parameters to achieve target oxygen level (default \$10%\$).

Step 7: Generate Reports (lines 101-106)

```
result.summaryText = TrialReportGenerator.GenerateSummaryReport(...);  
result.fullDetailsText = TrialReportGenerator.GenerateFullReport(...);  
// Summary: optimized parameters, prediction error  
// Full: coefficients, cross-validation, trial details
```

Step 8: Save Results (lines 117-127)

```
TrialRegressionAlgorithm.SaveRegressionResultsToFile(result);  
// Saves to: Application.persistentDataPath/RegressionResults/  
// Format: RegressionReport_YYYYMMDD_HHMMSS.txt
```

Result Storage

During Trials

Location: `Application.persistentDataPath/TrialResults/`

Files:

- `TrialParameters.csv` - Main trial data (parameters + results)
- `random_trial.csv` - Random trial configurations

Format: CSV with headers, one row per trial

After Regression Analysis

Location: `Application.persistentDataPath/RegressionResults/`

File naming: `RegressionReport_YYYYMMDD_HHMMSS.txt`

Content structure:

```
=== REGRESSION ANALYSIS SUMMARY ===  
Trials: 5 (3R+2Rand) | Target: 10.0% | Avg Error: 2.45%  
  
Optimized Parameters:  
speed: 23.45, verticalSpeed: 28.30, ...
```



```
Prediction: 10.12% | Error: 0.120%

=== FULL DETAILS ===
[Model coefficients]
[Cross-validation metrics]
[Trial-by-trial breakdown]
[Feature importance ranking]
```

Includes training error, R^2 score, cross-validation results, and optimized parameters.

Code location: `Assets/Scripts/TrialReportGenerator.cs` method `SaveToFile()` lines 330-369

Example: 5 Trials Analysis

Input: Patient completes 5 trials with oxygen results: 35%, 42%, 28%, 38%, 40% (average $\sim 36.6\%$)

Model Training: $\text{finalOxygen} = 50.2 - 0.85 \cdot \text{speed} - 0.42 \cdot \text{verticalSpeed} - \dots$
Training error: 2.3%, $R^2 = 0.92$

Optimization (target 10%):

- Phase 1: Analytical solution predicts 12.5%
- Phase 2: Gradient refinement achieves 10.08% (error 0.08%)

Output: Optimized parameters ready for next session, report saved to CSV.
